

Java

Strings, class & object

Theory Notes

1. Constructors

- Special methods used to initialize objects.
- Types:
 - **Default/Non-parameterized** → no arguments.
 - **Parameterized** → accepts arguments.
 - **Copy constructor** → initializes object using another object.

2. Inheritance

- **Single level** → one parent, one child.
- **Multilevel** → grandparent → parent → child.
- **Hierarchical** → one parent, multiple children.
- **Multiple** → multiple parents → one child (via interfaces).
- **Hybrid** → combination of multiple types.

3. Encapsulation

- Binding of data (fields) with methods.
- Achieved using `private` fields and `public` getters/setters.
- Ensures controlled access and data hiding.

4. Polymorphism

- **Compile-time polymorphism** → method overloading (same name, different parameters).
- **Runtime polymorphism** → method overriding (child class redefines parent method).

Coding Round Questions

1. Create a `Car` class with constructors and methods `start()` and `accelerate()`. Demonstrate object creation with different constructors.
2. Implement single-level inheritance with `Parent` and `Child` classes.
3. Demonstrate multilevel inheritance with `Grandpa` → `Parent` → `Child`.
4. Create a class `Encap` with private data and public getter/setter methods.
5. Write a program to show method overloading (`add(int a, int b)` and `add(int a, int b, int c)`).
6. Demonstrate method overriding with a `Payment` class and subclasses `UPI`, `Card`, `COD`.

Problem: E-Commerce Payment System

Requirements:

- Create a base class `Payment` with a method `pay()`.
- Subclasses: `UPI`, `Card`, `COD` each override `pay()` with their own implementation.
- Create a `Customer` class with fields `name`, `cartAmount`.
- Add a method `checkout(Payment mode)` that calls the appropriate payment method.
- Demonstrate polymorphism by passing different payment modes at runtime.

```
class Car{
    String color;
    int speed;

    Car(){ //constructor
        color ="Red";
        speed =40;
        System.out.println("Car constructor is called");
    }
    // types of constructor -> default/non-parameterized , parameterized

    Car(String color, int speed){
        this.color = color;
        this.speed = speed;
    }

    void start(){
        System.out.println("Car started");
    }

    void accelerate(){
        speed += 10;
    }
}

public class oops {

    public static void main(String[] args) {

        // Car obj = new Car();
        Car obj = new Car("Blue", 60);

        System.out.println(obj.color);
        obj.start();
        obj.accelerate();
    }
}
```

```

        System.out.println(obj.speed);
        // constructor -> special type of method
        Car obj1 = new Car("Green", 100);

        System.out.println(obj1.color);
        System.out.println(obj1.speed);

        System.out.println("OOPS");
        // object oriented programming
        // programming paradigm that organizes software design
        // around objects rather than functions

        // class & objects
        // four pillar -> inheritance, polymorphism, abstraction,
encapsulation

    }
}

// class Parent{
//     int speed = 10;

//     void display(){
//         System.out.println("Parent class");
//     }
// }

// class Child extends Parent {
//     void def(){
//         System.out.println("Child method");
//     }
// }

// class Child2 extends Parent{
//     void ghi(){
//         System.out.println("Child2 method");
//     }

```

```
// }

class Grandpa{
    void xyz(){
        System.out.println("Grandpa ");
    }
}

class Parent extends Grandpa{
    void abc(){
        System.out.println("parent ");
    }
}

class Child extends Parent{
    void def(){
        System.out.println("child");
    }
}

// single level inheritance one parent - one child
// multi level    - > grandparent -> parent -> child
// hierarchical    -> one parent -> multiple child

// multiple    -> multiple parents -> one child (through interface) will thought
in next class
// hybrid -> combination of multiple and hierarchical

public class inheritance {

    public static void main(String[] args) {

        System.out.println("Inheritance");

        // Child obj = new Child();

        // obj.def();
        // obj.display();
        // System.out.println(obj.speed);

        Child obj = new Child();
        obj.def();
        obj.abc();
        obj.xyz();
    }
}
```

```

    }

}

class Encap{
    private int data = 40;

    public void setData(int data){
        this.data = data;
    }

    public int  getData(){
        return data;
    }
}

public class Encapsulation {

    private int data = 10;

    public static void main(String[] args) {
        System.out.println("Encapsulation");

        Encapsulation obj = new Encapsulation();

        System.out.println(obj.data);

        Encap obj1 = new Encap();

        obj1.setData(60);
        int result = obj1.getData();
        System.out.println(result);

        // encapsulation -> binding of data (field/variables) with method
    }
}

class Operator{

```

```
    void add(int a, int b){
        System.out.println(a+b);
    }

    void add(int a, int b, int c){
        System.out.println(a+b+c);
    }
}

class Payment{

    void pay(){
        System.out.println("Payment is done");
    }
}

class UPI extends Payment{
    void pay(){
        System.out.println("UPI payment successfull");
    }
}

class Card extends Payment{
    void pay(){
        System.out.println("Card payment successfull");
    }
}

class COD extends Payment{
    void pay(){
        System.out.println("Cash on delivery");
    }
}

public class Polymorphism {

    public static void main(String[] args) {
        System.out.println("Polymorphism");

        // poly -> many forms

        // same method different implementation

        // polymorphism -> compile time, runtime
    }
}
```

```
// compile time -> method overloading ( same name with different  
parameter)  
// runtime -> method overriding ( same name overrides in subclasses)  
  
Operator obj = new Operator();  
  
obj.add(10,20);  
  
obj.add(40,50,60);  
  
Payment obj1 = new UPI();  
Payment obj2 = new COD();  
  
obj1.pay();  
obj2.pay();  
}  
  
}
```